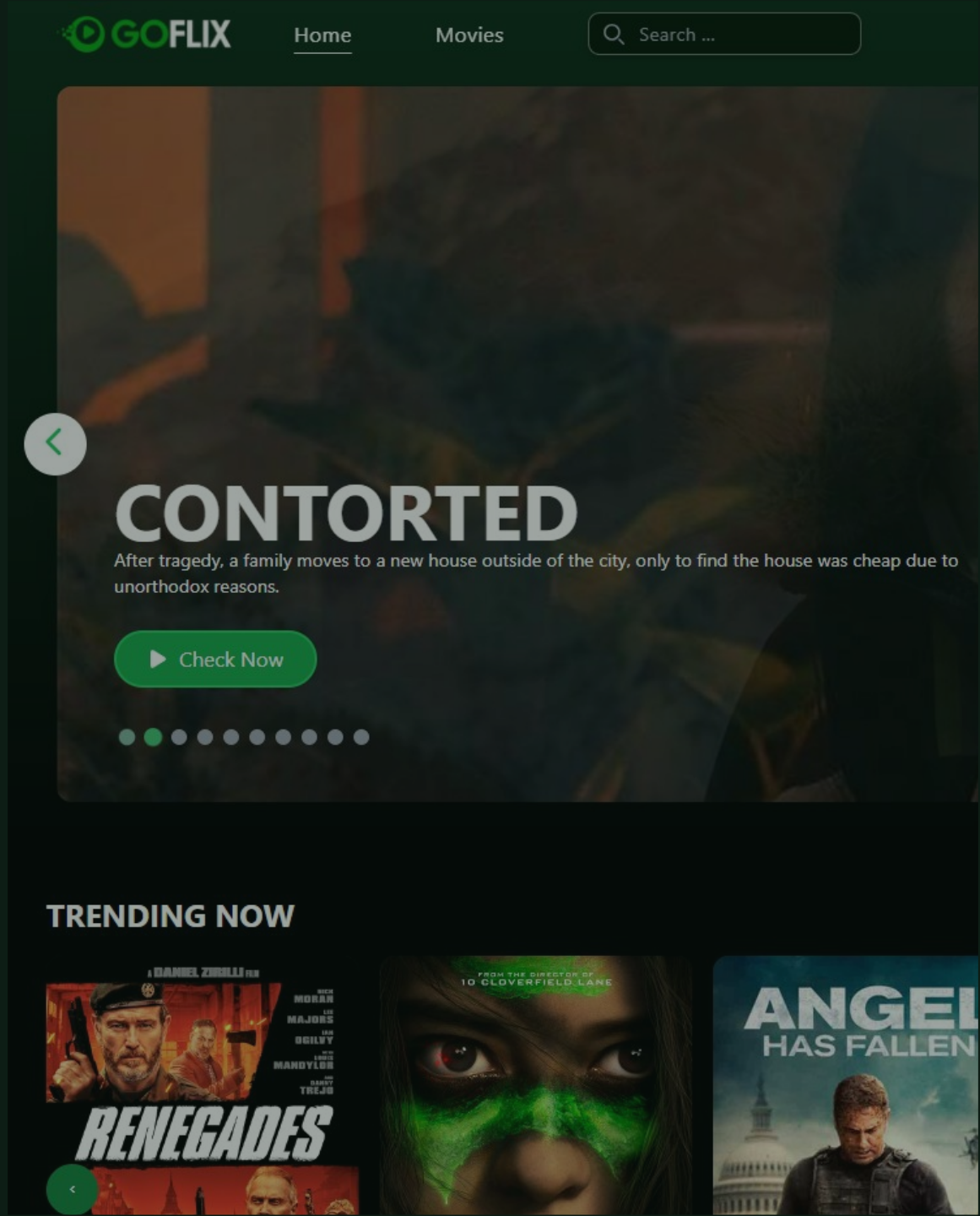
STREAMING PLATFORM

Goflix

The admin dashboard behind the catalog, the plans, and who may watch what

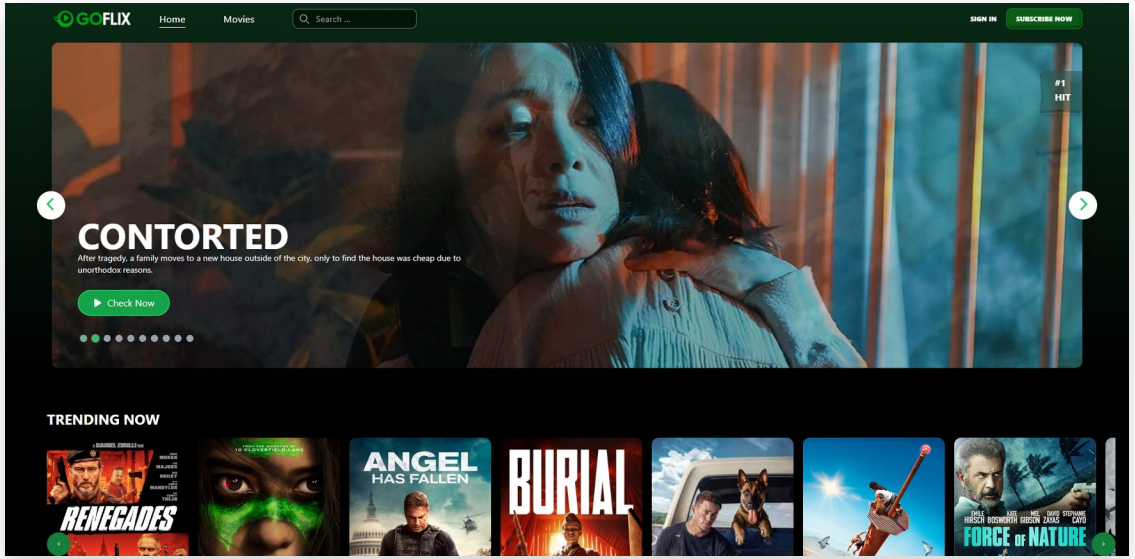
Enrico Saputra · Backend developer, Laravel dashboard

Client code and data are private. The demo rebuilds the Laravel dashboard on the real data model, with made-up data.

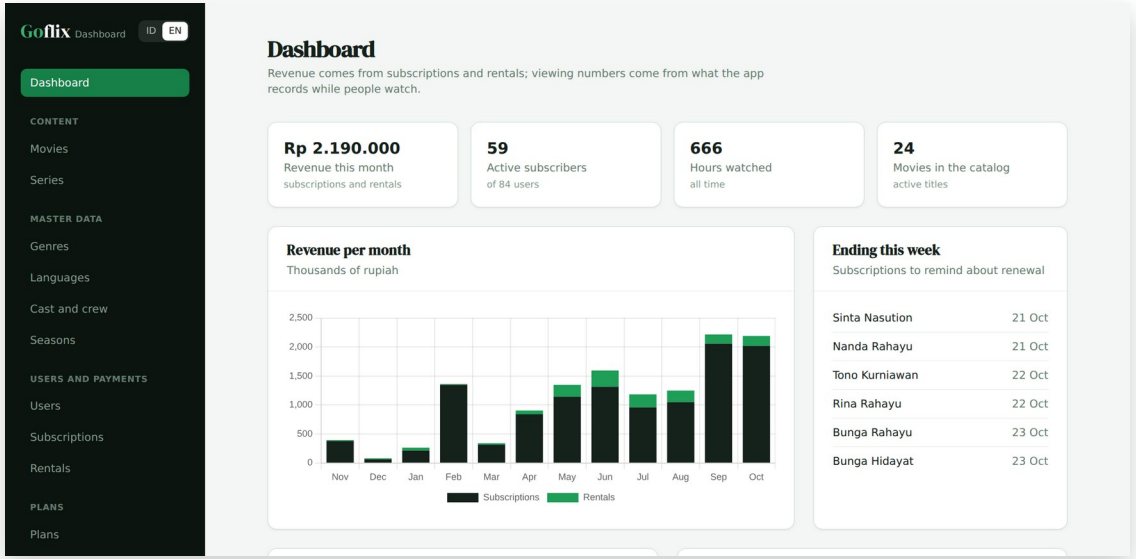


Viewers see this; the team works here

01



Viewer app · Vue + Go



Dashboard · Laravel · my part

The content team adds movies and episodes, the business team sets plans and coupons, and support answers "why can't I watch this?"

Three ways to watch

02



Free

Free titles play for anyone with an active account.



Plan

A monthly or yearly plan opens every premium title.



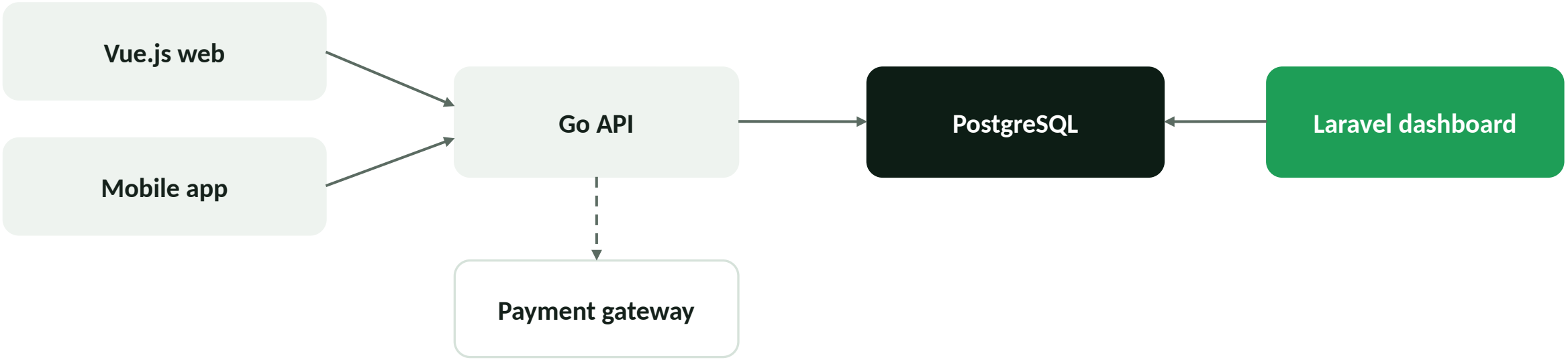
Rental

One premium movie, for a few days, without a plan.

My part: the catalog, plans and coupons, users and payments, and the reports on them.

Three parts, one database

03



Two writers

The API writes activity and in-app payments; the dashboard writes the catalog, plans and corrections.

The server decides

The app never decides access on its own. The dashboard shows support the same rule.

No API in the dashboard

Everything the apps need goes through the Go API.

The data model in three groups

04

Catalog

- movies
- shows
- episodes
- seasons
- genres
- languages
- people

Users and money

- users
- packages
- package_features
- subscriptions
- rentals
- coupons

Activity

- watch_history
- bookmarks

Written by the apps while people watch. The dashboard only reads it, for reports and continue watching.

users.subscription_ends_on is the source of truth for a fast access check; only SubscriptionService may change it.

Relations: from text to pivot tables

05

Production

```
video
  id: 12
  category_id: '6,12,19'
  cast_id: '1,108,109,110'
```

Cannot be joined or indexed; deleted ids stay behind.

Demo

```
genre_movie
  (movie_id 12, genre_id 6)
  (movie_id 12, genre_id 12)
  (movie_id 12, genre_id 19)
```

Foreign keys, normal joins, and the genre report is one query.

Gradual migration: add the pivot, fill it from the old column, move readers one by one, then drop the old column.

Renewing without losing paid days

06

Active until 20 Mar, buys 12 months on 10 Mar



Ended 31 Jan, buys 1 month on 10 Mar



One place

Only SubscriptionService moves users.subscription_ends_on.

One transaction

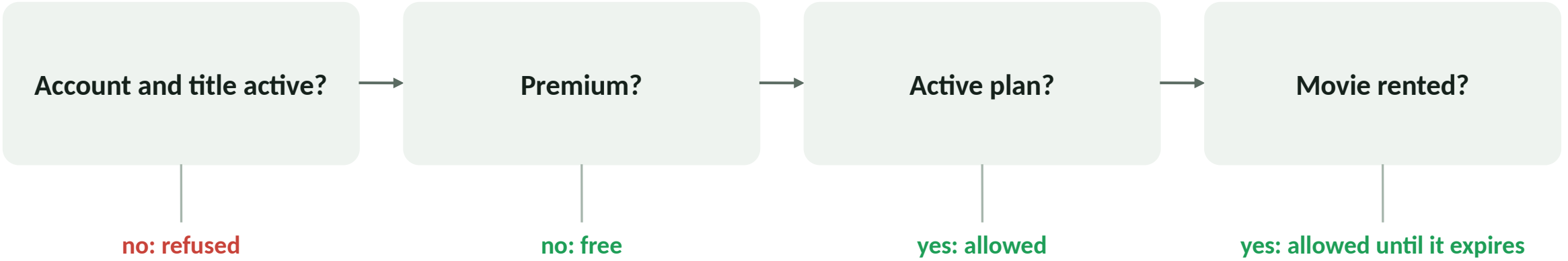
The payment and the end date are saved together, or neither is.

A row lock

lockForUpdate makes a second payment wait and count from the new date.

Who may watch what

07



Refused: needs a plan, or a rental if the movie has a rent price

An episode is premium on its own or through its series. One AccessService is used everywhere, and the dashboard shows it per user so support can answer complaints.

Rentals and coupons

08

Renting one movie

- Only premium movies with a rent price
- Lasts rent_days days from payment
- Active subscribers cannot rent: it is already included
- The same movie cannot be rented twice at once

Coupons

- Fixed amount or percentage
- Only valid between its start and end dates
- Never takes a price below zero
- A single-use coupon works once per user

Example: an active subscriber tries to rent a premium movie → refused, because the plan already includes it. Nobody pays twice, and refund requests drop.

The trade-offs I chose

09

Decision	Why	What it costs
subscription_ends_on is stored on users	The API checks access on every play; one indexed column is enough	Duplicated data; only SubscriptionService may change it
Payments cannot be edited	Plans and rentals are financial records	A correction has to be a new entry
Deactivate, not delete	Genres, plans or users in use keep their history	Lists need an active filter
Episodes updated in place	Viewing history stays attached to the same rows	Saving takes more code than delete-and-recreate

Why this stack, and what else I considered

10

Choice	Why	What I considered instead
Laravel 10	The dashboard: forms, validation, CRUD. My part	Go for the dashboard too: one language, but admin CRUD takes much longer to build
PostgreSQL	Shared with the Go API; transactions and row locks	A separate database for the dashboard: two-way syncing
Go	The apps' API: heavy playback traffic	Not my part; it existed before the dashboard
Blade + Chart.js	An internal admin needs no SPA; one chart needs only Chart.js	Vue like the viewer web: a separate build and state for form pages

A renewal is one transaction

11

```
return DB::transaction(function () use (...) {
    $user = User::lockForUpdate()
        ->findOrFail($user->id);
    $startsOn = $user->subscription_ends_on
        ?->gte($today)
        ? $user->subscription_ends_on->addDay()
        : $today;
    $endsOn = $startsOn->copy()
        ->addMonthsNoOverflow($months)->subDay();
    $user->subscriptions()->create([...]);
    $user->update(['subscription_ends_on' => $endsOn]);
});
```

Rules in services

Thin controllers; plans, rentals, coupons and access are plain classes.

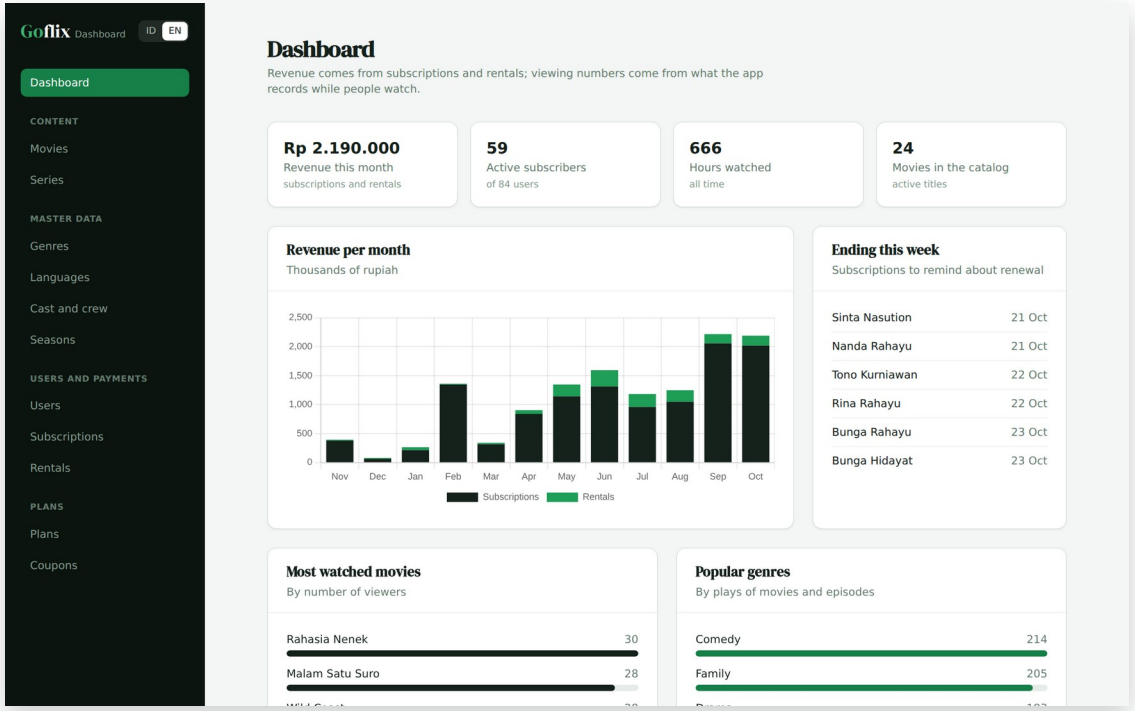
Enums, not numbers

Sign-in, role and coupon type are typed.

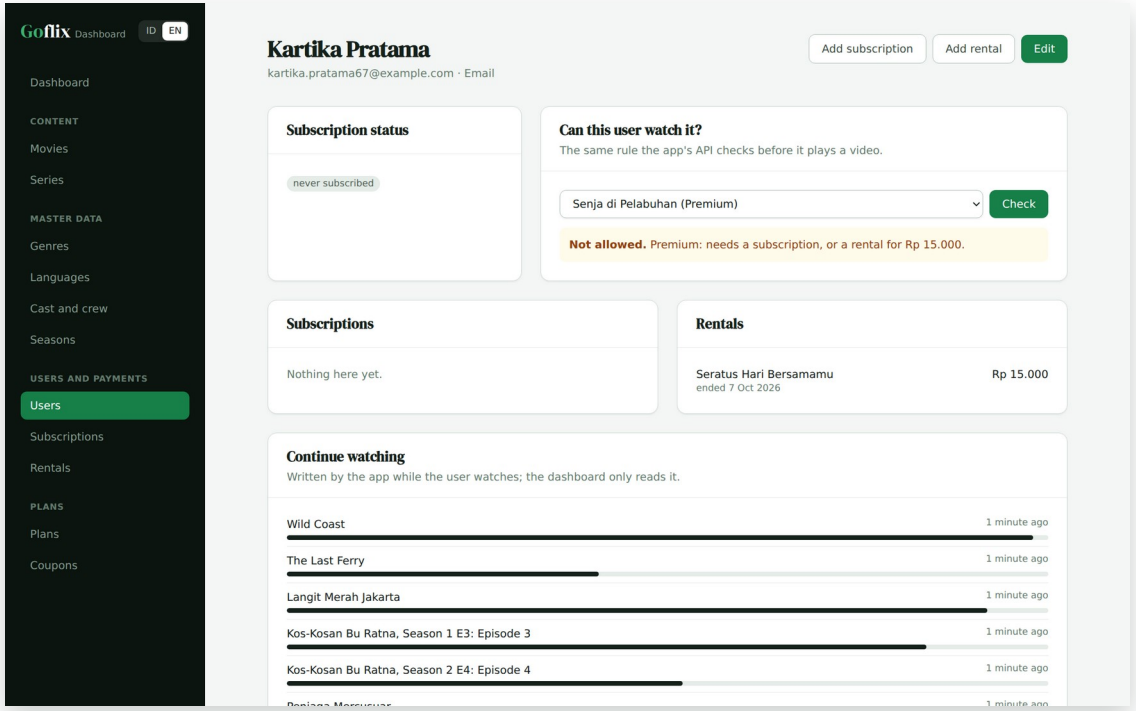
Tests on the rules

33 feature tests.

Demo: revenue, then a user's access check



Dashboard: plan and rental revenue per month



User page: status, payments and the access check

What is not done yet

13

Not done yet	First step
Production still stores comma-separated ids	Add the pivot tables, fill them from the old columns, move readers one by one
The access rule lives in Go and in Laravel	One shared set of test cases that both sides run
watch_history keeps growing	Partition by month; reports from a daily summary table
Production has no foreign keys	Foreign keys and indexes on the access-check and report columns

Demo and slides

Live demo	https://goflix-dashboard-demo.vercel.app
Slides (PDF)	https://goflix-dashboard-demo.vercel.app/goflix-walkthrough.pdf
Slides (PPTX)	https://goflix-dashboard-demo.vercel.app/goflix-walkthrough.pptx

The demo code is in a private GitHub repo; access is given during the interview. The dashboard speaks Bahasa Indonesia and English; demo data is restored every 3 hours.